

Two-dimensional lists

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	Two-dimensional lists (lesson 8 of the unit)
Lesson ID	pr-108
Experience ID	pr-108
Plan ID	PLAN-PR-L08
Content version	bf722d96dbe1
Website	https://revise360.co.uk/experience.html?id=pr-108

Big question, outcomes and success criteria

Big question — Which index comes first when a program reads a cell, and why does swapping the two over give a different answer?

Prerequisites

The lesson before this one is pr-107. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-101, pr-102, pr-103, pr-104, pr-105, pr-106.

Assessable outcomes

- pr-108-o1** — know how to build a two-dimensional list as a list of lists and fill it from input
- pr-108-o2** — understand how a row index and a column index together pick out one cell
- pr-108-o3** — know how to write nested loops that print a grid and total any row or column of it

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- The user types in four lines, one whole number on each. **pr-108-s1-a5**
- Store them as whole numbers, not as the text that comes back. **pr-108-s1-a5**
- Displaying the grid shows both sets of square brackets, so `[[4, 8], [6, 2]]` is right. **pr-108-s1-a5**
- Do not change the six numbers. **pr-108-s1-a6**
- Displaying the grid shows both sets of square brackets, so `[[4, 9, 2], [7, 8, 1]]` is right. **pr-108-s1-a6**
- The user types in six lines, one word on each. **pr-108-s1-a7**
- Displaying a grid of words shows quotation marks round each one, so `[[ 'red', 'blue'], [ 'grey', 'pink'], [ 'teal', 'navy']]` is right. **pr-108-s1-a7**
- Three lines of output: the grid, then the two counts. **pr-108-s1-a7**

Vocabulary, resources and preparation

Key vocabulary — Two-dimensional list, Row and column, Cell, Nested loop

Resources

SLIDES [presentations/PR_L08_TwoDimensionalLists_Lesson.pptx](#)

WORKSHEET	worksheets/PR_L08_TwoDimensionalLists_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=pr-108
CODE BANK	tools/codebank/pr-108.json

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>PR_L08_TwoDimensionalLists_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. A list of lists

- A two-dimensional array is a list whose items are themselves lists.
- Each inner list is one row, and the rows normally all hold the same number of items.
- Common mistake: expecting `len()` on a grid to count the cells, when it counts the rows.

Challenge on this wall: How many rows and how many columns does `[[1, 2, 3], [4, 5, 6]]` have?

2. Rows and columns

- `grid[row][col]` reads one cell, with the row first and the column second.
- `grid[row]` on its own gives back that whole row as a list.
- Common mistake: swapping the two indexes over, which reads a completely different cell.

Challenge on this wall: Which index comes first when you read a cell, the row or the column?

3. Going through a grid

- Two nested loops visit every cell: the outer picks the row, the inner the column.
- Print a new line after the inner loop ends, to start the next row.
- Common mistake: putting that new line inside the inner loop, which prints one value per line.

Challenge on this wall: Which loop should be the outer one if you want to print row by row?

4. Totalling a row

- To total one row, loop across the columns with the row index held still.
- `sum(grid[r])` does the same job in one step, once you know which row you want.
- Common mistake: letting the row index change inside the loop, which totals the wrong cells.

Challenge on this wall: Which index stays the same all the way through a row total?

5. Totalling a column

- To total one column, loop down the rows with the column index held still.
- The loop runs once for every row, so its range is `len(grid)`.
- Common mistake: using the number of columns as the range when travelling down a column.

Challenge on this wall: Why does a column total loop over the rows rather than the columns?

6. Putting it together

- To search a grid, loop every cell and record the row and column where you found it.
- A found flag, set before the loops start, says whether anything matched at all.
- Common mistake: reporting only the row when the task asks where in the row it was too.

Challenge on this wall: How would you find a word in a grid and report both the row and the column it sits at?

The example to model

A grid is a list whose items are themselves lists. Each item is one row. Writing it over several lines, one row to a line, shows its shape.

```
grid = [[4, 9, 2],
        [7, 8, 1]]
print(grid)
print(len(grid))
```

It displays `[[4, 9, 2], [7, 8, 1]] / 2`

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — The grid `g` holds two rows: 4, 9, 2 in the first and 7, 1, 6 in the second. Write down what `g[0][2]`, `g[1][0]` and `len(g)` each give, and which row and column hold the 6.

Answer space: 4 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-108-s4-c2), so the wording a pupil sees matches the wording you say.

What will this program display?

Correct: 15 3

What a pupil might choose instead

- 16 3
- 15 2

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: Row 0 is 4, 9 and 2, which comes to 15, and that row holds three values. `len(grid)` would have said 2.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- expecting `len()` on a grid to count the cells, when it counts the rows. [wall panel](#)
- Count the items the grid has at the moment. A grid of two rows should have two. [pr-108-s1-a6, a Fix it activity](#)
- swapping the two indexes over, which reads a completely different cell. [wall panel](#)
- A row of this grid holds two values, so one of these two numbers is being used in the wrong place. [pr-108-s2-a6, a Fix it activity](#)
- putting that new line inside the inner loop, which prints one value per line. [wall panel](#)
- Look at what the inner loop is being given to walk along. [pr-108-s3-a6, a Fix it activity](#)
- letting the row index change inside the loop, which totals the wrong cells. [wall panel](#)
- Row 1's total has row `0` still inside it. Where is the total set to `0`? [pr-108-s4-a6, a Fix it activity](#)

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Guided	22	Support is in the activity: an example to run, an output to predict, one thing to change, a gap to fill.
Independent	15	The pupil writes or fixes it with the brief and nothing else.

Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.
-----------	---	---

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	A list of lists	7	A grid is stored much the way a database table is, with every row holding one list of...
2	Rows and columns	7	The exam writes a cell as <code>array[1,2]</code> , where Python writes that very same cell as...
3	Going through a grid	7	A grid of 10 rows by 10 columns holds 100 cells, so a nested loop over it runs...
4	Totalling a row	7	Add up every row total and the number you are left with is the total of...
5	Totalling a column	7	A row total and a column total are the same few lines with the two indexes...
6	Putting it together	7	Checking every cell of a grid in turn is really a linear search, only in...
final	Final challenge: read the grid	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-108-x1	A program that should total column 2 of a grid of 4 rows by 3 columns begins its loop with <code>for r in range(len(grid[0]))</code> . Explain why this looks at the wrong number of cells, and write the line corrected.	3	worksheet, Exam practice a); deck, Exam practice slide
pr-108-x2	Write a program that reads twelve whole numbers into a grid of three rows of four, then prints the total of each row on a line of its own.	5	worksheet, Exam practice b); deck, Exam practice slide
pr-108-x3	A grid of 2 rows by 3 columns is already stored in a program. Write the code that prints it one row to a line, with single spaces between the values and no brackets.	4	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-108.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-108.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — red blue `[['red', 'blue'], ['grey', 'pink']]; ['grey', 'pink']` `[['red', 'blue'], ['grey', 'pink']]`
- **What will this program display?** — 9 7; 7 8
- **What will this program display?** — 4 7 9 8; `[4, 9]` `[7, 8]`
- **What will this program display?** — 16 3; 15 2
- **What will this program display?** — `[5, 2]` `[3, 6]; [3, 6]` `[3, 5, 9]`

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Building a list of lists and filling it
- Reading a cell with a row index and a column index

- Totalling rows and columns with nested loops

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-109, Subprograms. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 43 activities across 6 stations, and a beginner does not finish 43 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.